

Modelado Relacional

Entidad-Relación, Cardinalidad y Normalización

Motor: SQLite · Entorno: Google Colab · Docente: Pilar Rocío Sayán Mejía

Contenido

01 ¿Qué es el Modelo Relacional?	02 Modelo Entidad-Relación (ER)
03 Cardinalidad: 1:1, 1:N y N:M	04 Normalización: 1FN, 2FN y 3FN
05 De diagrama ER a tablas SQLite	06 Referencias bibliográficas
07 Lab D4: Diseño ER del proyecto grupal	08 Checklist de entrega Lab D4

01 ¿Qué es el Modelo Relacional?

El modelo relacional es la base teórica de todos los sistemas de bases de datos SQL. Fue propuesto por Edgar F. Codd en 1970 en IBM. La idea central es organizar los datos en tablas (relaciones) formadas por filas y columnas, conectadas mediante claves.

Concepto	En el modelo relacional	En SQLite (práctica)
Relación	Tabla con filas y columnas	CREATE TABLE empleados (...)
Tupla	Una fila de la tabla	INSERT INTO empleados VALUES (...)
Atributo	Una columna de la tabla	nombre TEXT NOT NULL
Dominio	Tipo de dato permitido	INTEGER, TEXT, REAL...
Clave primaria	Identifica cada fila única	id INTEGER PRIMARY KEY
Clave foránea	Conecta dos tablas	FOREIGN KEY (id_dep) REFERENCES ...
Integridad referencial	Un FK debe apuntar a un PK existente	Garantizada por FOREIGN KEY

¿Por qué aprender el Modelo Relacional?

Antes de escribir SQL, debes saber QUE vas a construir.

El diseño determina qué consultas serán fáciles o imposibles.

Un buen modelo = menos código SQL, menos errores, más velocidad.

02 Modelo Entidad-Relación (ER)

El Modelo ER, creado por Peter Chen en 1976, es la herramienta estándar para diseñar bases de datos antes de implementarlas. Tiene tres elementos: entidades, atributos y relaciones.

2.1 Entidades

Entidad	¿Qué representa?	Tabla en SQLite
CLIENTE	Una persona o empresa que compra	CREATE TABLE clientes (...)
PRODUCTO	Un artículo que se vende	CREATE TABLE productos (...)
EMPLEADO	Una persona que trabaja en la empresa	CREATE TABLE empleados (...)
PEDIDO	Una transacción de compra	CREATE TABLE pedidos (...)
DEPARTAMENTO	Un área de la empresa	CREATE TABLE departamentos (...)

2.2 Atributos

Tipo de atributo	Descripción	Ejemplo
Clave (PK)	Identifica de forma única cada fila	id_cliente (subrayado en diagrama)
Simple	Un solo valor, indivisible	nombre, precio, fecha
Compuesto	Se puede dividir en partes	dirección = calle + número + ciudad
Multivalorado	Puede tener varios valores	teléfonos (varios por cliente)
Derivado	Se calcula de otro atributo	edad (calculada de fecha_nacimiento)
Nulo (NULL)	Puede no tener valor	email (opcional)

2.3 Relaciones — Diagrama ER del Sistema AyniTech

N **1** **N** **M**
 CLIENTE ----realiza----> PEDIDO ----contiene----> PRODUCTO
 id_cliente, nombre, ciudad id_pedido, fecha id_prod, precio, nombre

 Rectángulos = entidades | Óvalos = atributos | Rombos = relaciones
 Números (1, N, M) = cardinalidad de cada extremo

03 Cardinalidad: 1:1, 1:N y N:M

La cardinalidad indica cuántas instancias de una entidad pueden relacionarse con cuántas de otra. Es la decisión de diseño más importante porque determina cómo se implementan las tablas en SQL.

3.1 Relación 1:1 — Uno a uno

Ejemplo real	Entidad A	Entidad B	Interpretación
Empleado ↔ Credencial	EMPLEADO	CREDENCIAL	Cada empleado tiene una sola credencial
País ↔ Capital	PAIS	CIUDAD_CAPITAL	Cada país tiene una sola capital
Persona ↔ Pasaporte	PERSONA	PASAPORTE	Cada persona tiene un solo pasaporte

```
-- Relación 1:1: Empleado ↔ Credencial
CREATE TABLE empleados (
    id_emp INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL
);
CREATE TABLE credenciales (
    id_cred INTEGER PRIMARY KEY,
    codigo TEXT UNIQUE NOT NULL,
    id_emp INTEGER UNIQUE NOT NULL, -- UNIQUE asegura la relación 1:1
    FOREIGN KEY (id_emp) REFERENCES empleados(id_emp)
);
```

3.2 Relación 1:N — Uno a muchos

La FK siempre va en el lado N (el lado "muchos").

Ejemplo real	Lado 1 (padre)	Lado N (hijo)	FK va en...
Departamento → Empleados	DEPARTAMENTO	EMPLEADO	EMPLEADO
Cliente → Pedidos	CLIENTE	PEDIDO	PEDIDO
Categoría → Productos	CATEGORIA	PRODUCTO	PRODUCTO
Profesor → Cursos	PROFESOR	CURSO	CURSO

```
-- Relación 1:N: Departamento → Empleados
CREATE TABLE departamentos ( -- Tabla PADRE (lado 1)
    id_depto INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL UNIQUE
);
CREATE TABLE empleados ( -- Tabla HIJA (lado N)
    id_emp INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    id_depto INTEGER NOT NULL, -- FK apunta al padre
    FOREIGN KEY (id_depto) REFERENCES departamentos(id_depto)
);
```

3.3 Relación N:M — Muchos a muchos

Se resuelve creando una tabla intermedia (tabla pivote) con las FK de ambas tablas.

Ejemplo real	Entidad A	Entidad B	Tabla intermedia
Estudiantes ↔ Cursos	ESTUDIANTE	CURSO	matriculas (id_est, id_cur)
Pedidos ↔ Productos	PEDIDO	PRODUCTO	detalle_pedido (id_ped, id_pro)
Médicos ↔ Pacientes	MEDICO	PACIENTE	consultas (id_med, id_pac)
Actores ↔ Películas	ACTOR	PELICULA	actuaciones (id_act, id_pel)

```
-- Relación N:M: Pedidos ↔ Productos
CREATE TABLE detalle_pedido ( -- Tabla INTERMEDIA
    id_det          INTEGER PRIMARY KEY AUTOINCREMENT,
    id_pedido       INTEGER NOT NULL,    -- FK → pedidos
    id_producto     INTEGER NOT NULL,    -- FK → productos
    cantidad        INTEGER NOT NULL DEFAULT 1,
    precio_unitario REAL NOT NULL,      -- Precio en el momento de compra
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido),
    FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
);
```

Atributos de la relación N:M:

La tabla intermedia puede tener atributos propios que pertenecen a la RELACIÓN, no a ninguna de las dos entidades individuales.

Ejemplo: "cantidad" y "precio_unitario" pertenecen a la relación entre ese pedido específico y ese producto específico.

04 Normalización: 1FN, 2FN y 3FN

La normalización elimina la redundancia y las dependencias problemáticas. Cada Forma Normal es una regla más estricta que la anterior.

4.1 Primera Forma Normal (1FN)

Cada celda tiene un solo valor atómico, no hay grupos repetitivos, y existe una clave primaria.

id_cliente	nombre	telefonos	productos_comprados
1	María Quispe	984111222, 984333444	Office, Antivirus, Soporte
2	Carlos Ttito	984555666	Laptop, Disco

□ La tabla anterior viola 1FN. La solución:

```

-- Tabla clientes normalizada (1FN)
CREATE TABLE clientes (
    id_cliente INTEGER PRIMARY KEY,
    nombre     TEXT NOT NULL
);
-- Los teléfonos en tabla separada (resuelve el multivalorado)
CREATE TABLE telefonos (
    id_tel      INTEGER PRIMARY KEY,
    numero      TEXT NOT NULL,
    id_cliente  INTEGER NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);

```

4.2 Segunda Forma Normal (2FN)

1FN + todos los atributos no-clave dependen de la PK COMPLETA. Solo aplica cuando la PK es compuesta.

```

-- Tabla detalle_pedido: solo atributos de la RELACIÓN
CREATE TABLE detalle_pedido (
    id_pedido  INTEGER,
    id_producto INTEGER,
    cantidad   INTEGER NOT NULL,      -- Depende de AMBAS claves
    PRIMARY KEY (id_pedido, id_producto), -- PK compuesta
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido),
    FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
);
-- Tabla productos: atributos del PRODUCTO
CREATE TABLE productos (
    id_producto INTEGER PRIMARY KEY,
    nombre       TEXT NOT NULL,      -- Depende de id_producto
    precio       REAL NOT NULL       -- Depende de id_producto
);

```

4.3 Tercera Forma Normal (3FN)

2FN + ningún atributo no-clave depende de otro atributo no-clave (sin dependencias transitivas).

```
-- Tabla empleados: solo datos del empleado
CREATE TABLE empleados (
    id_emp      INTEGER PRIMARY KEY,
    nombre      TEXT NOT NULL,
    cod_postal  TEXT NOT NULL, -- Solo el código postal
    FOREIGN KEY (cod_postal) REFERENCES ciudades(cod_postal)
);

-- Tabla ciudades: relación postal → nombre
CREATE TABLE ciudades (
    cod_postal  TEXT PRIMARY KEY,
    nombre      TEXT NOT NULL -- Ahora en un solo lugar
);
```

Forma Normal	Regla que debe cumplir	¿Qué elimina?
1FN	Un solo valor por celda. Sin grupos repetitivos. Tiene PK.	Celdas con múltiples valores
2FN	Cumple 1FN + cada atributo depende de la PK COMPLETA.	Dependencias parciales de la PK
3FN	Cumple 2FN + ningún atributo depende de otro no-clave.	Dependencias transitivas
BCNF	Cada determinante es clave candidata. (Nivel avanzado)	Anomalías residuales de la 3FN

05 De diagrama ER a tablas SQLite en Google Colab

Reglas de mapeo ER → SQL:

1. Cada ENTIDAD → una tabla en plural
2. Cada ATRIBUTO → una columna con su tipo
3. Atributo CLAVE → PRIMARY KEY (+ AUTOINCREMENT)
4. Relación 1:N → FOREIGN KEY en la tabla del lado N
5. Relación N:M → tabla intermedia con las dos FK
6. Relación 1:1 → FOREIGN KEY + UNIQUE en tabla secundaria
7. Atributo MULTIVALORADO → tabla separada con FK

Caso completo: Sistema de Matrícula Universitaria

Entidades: ESTUDIANTE, CURSO, PROFESOR. Relaciones: ESTUDIANTE N:M CURSO (via MATRICULA) | PROFESOR 1:N CURSO

Celda 1 — Conexión

```

import sqlite3
from tabulate import tabulate
conn = sqlite3.connect('universidad.db')
cur = conn.cursor()
def ver(q):
    cur.execute(q)
    f=cur.fetchall(); c=[d[0] for d in cur.description]
    print(tabulate(f, headers=c, tablefmt='grid'))
    print(f'{len(f)} fila(s)\n')

```

Celda 2 — Crear tablas siguiendo el mapeo ER

```

-- 1. PROFESOR (sin FK, va primero)
CREATE TABLE IF NOT EXISTS profesores (
    id_profesor    INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre         TEXT NOT NULL,
    especialidad   TEXT NOT NULL,
    email          TEXT UNIQUE
);

-- 2. CURSO (FK → profesores: relación 1:N)
CREATE TABLE IF NOT EXISTS cursos (
    id_curso       INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre         TEXT NOT NULL,
    creditos       INTEGER NOT NULL DEFAULT 4,
    id_profesor    INTEGER NOT NULL, -- FK → profesores (lado N)
    FOREIGN KEY (id_profesor) REFERENCES profesores(id_profesor)
);

-- 3. ESTUDIANTE (sin FK)
CREATE TABLE IF NOT EXISTS estudiantes (
    id_estudiante  INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre         TEXT NOT NULL,
    codigo         TEXT UNIQUE NOT NULL,
    ciudad         TEXT DEFAULT 'Cusco'
);

-- 4. MATRICULA (tabla intermedia: N:M)
CREATE TABLE IF NOT EXISTS matriculas (
    id_matricula   INTEGER PRIMARY KEY AUTOINCREMENT,
    id_estudiante  INTEGER NOT NULL,
    id_curso       INTEGER NOT NULL,
    fecha_matricula TEXT NOT NULL,
    nota_final     REAL DEFAULT NULL,
    FOREIGN KEY (id_estudiante) REFERENCES estudiantes(id_estudiante),
    FOREIGN KEY (id_curso) REFERENCES cursos(id_curso),
    UNIQUE (id_estudiante, id_curso)
);
conn.commit()
print('Base de datos universidad creada')

```

Celda 3 — Insertar datos de prueba

```
INSERT INTO profesores (nombre, especialidad, email) VALUES
('Pilar Rocio Sayan Mejia', 'Data Science', 'psayan@tecsup.pe'),
('Carlos Mamani',          'Bases de Datos', 'cmamani@tecsup.pe'),
('Ana Quispe',             'Programacion', 'aquispe@tecsup.pe');

INSERT INTO cursos (nombre, creditos, id_profesor) VALUES
('Fundamentos de Gestion de Datos', 4, 1),
('Mineria de Datos', 4, 1),
('SQL Avanzado', 3, 2),
('Python para Datos', 4, 3);

INSERT INTO estudiantes (nombre, codigo, ciudad) VALUES
('Luis Ttito', '2024001', 'Cusco'),
('Rosa Huanca', '2024002', 'Cusco'),
('Marco Apaza', '2024003', 'Puno'),
('Diana Flores', '2024004', 'Cusco');

INSERT INTO matriculas (id_estudiante, id_curso, fecha_matricula) VALUES
(1, 1, '2024-03-01'), -- Luis → Fundamentos
(1, 3, '2024-03-01'), -- Luis → SQL Avanzado
(2, 1, '2024-03-02'), -- Rosa → Fundamentos
(4, 2, '2024-03-04'), -- Diana → Minería
(4, 3, '2024-03-04'); -- Diana → SQL Avanzado
conn.commit()
```

Celda 4 — Consultar el modelo completo

```
ver('''
SELECT e.nombre AS Estudiante,
       c.nombre AS Curso,
       p.nombre AS Profesor,
       m.fecha_matricula AS Matriculado
FROM   matriculas m
INNER JOIN estudiantes e ON m.id_estudiante = e.id_estudiante
INNER JOIN cursos      c ON m.id_curso      = c.id_curso
INNER JOIN profesores  p ON c.id_profesor   = p.id_profesor
ORDER BY e.nombre, c.nombre;
''')
```

06 Referencias bibliográficas recomendadas

Libro	Autor(es)	Editorial	Temas clave
SQL and Relational Theory (3rd Ed.)	C.J. Date	O'Reilly	Modelo relacional, álgebra relacional, normalización
Database Administration (2nd Ed.)	Craig S. Mullins	Addison-Wesley	Diseño de esquemas, normalización práctica

Libro	Autor(es)	Editorial	Temas clave
Learning SQL (3rd Ed.)	Alan Beaulieu	O'Reilly	SQL desde cero: SELECT, JOIN, GROUP BY
Database Design for Mere Mortals	Michael J. Hernandez	Addison-Wesley	Diseño ER accesible, cardinalidad, tablas de unión
SQLite Pocket Reference	Jay A. Kreibich	O'Reilly	Referencia rápida de SQLite

Acceso a los libros:

O'Reilly Learning: learning.oreilly.com — suscripción mensual, prueba 10 días.

Google Books: books.google.com — vista previa gratuita de capítulos.

Internet Archive: archive.org — ediciones antiguas en préstamo digital gratuito.

Biblioteca digital institucional: muchas universidades peruanas tienen ProQuest.

07 Lab D4: Diseño ER del proyecto grupal

Objetivo: El equipo diseña el diagrama ER de su proyecto, lo normaliza hasta 3FN

e implementa en SQLite desde Google Colab con datos de prueba reales.

Al finalizar, cada grupo tendrá una base de datos funcional en su Colab.

Paso 1 — Identificar entidades, atributos y relaciones

Pregunta guía	Ejemplo (proyecto Interbank)
¿Qué objetos queremos registrar?	Cliente, Tarjeta, Transacción, Comercio
¿Qué datos guardamos de cada objeto?	Cliente: DNI, nombre, email, ciudad
¿Cómo se relacionan los objetos?	Un cliente TIENE muchas tarjetas (1:N)
¿Cuál es la cardinalidad de cada relación?	Tarjeta N:M Comercio (via Transacción)
¿Qué atributo identifica cada entidad?	id_cliente, id_tarjeta, id_transaccion
¿Qué atributos son obligatorios (NOT NULL)?	DNI, nombre, número de tarjeta
¿Qué atributos son únicos (UNIQUE)?	email del cliente, número de tarjeta

Paso 2 — Verificar la normalización

```
-- CHECKLIST DE NORMALIZACIÓN
-- 1FN: ¿Cada celda tiene un solo valor?
--      ¿Guardamos varios emails en un campo? → separar en tabla telefonos

-- 2FN: ¿Todos los atributos dependen de la PK completa?
--      Si la PK es compuesta, ¿hay atributos que dependen de solo una parte?
--      Si SÍ → moverlos a una tabla separada

-- 3FN: ¿Hay atributos no-clave que dependen de otro atributo no-clave?
--      Ejemplo: ciudad depende de cod_postal, no de id_cliente
--      Si SÍ → crear tabla ciudades(cod_postal PK, nombre_ciudad)
```

Paso 3 — Plantilla base para Google Colab

```
import sqlite3
from tabulate import tabulate
# CAMBIAR ESTE NOMBRE POR EL DEL PROYECTO
conn = sqlite3.connect('proyecto_grupal.db')
cur = conn.cursor()

# TABLA 1: entidad sin FK (va primero)
cur.execute('''
CREATE TABLE IF NOT EXISTS entidad_padre (
    id_padre    INTEGER PRIMARY KEY AUTOINCREMENT,
    atributo1   TEXT      NOT NULL,
    atributo2   TEXT      UNIQUE,
    atributo3   REAL      DEFAULT 0.0
)''')

# TABLA 2: entidad con FK
cur.execute('''
CREATE TABLE IF NOT EXISTS entidad_hija (
    id_hijo     INTEGER PRIMARY KEY AUTOINCREMENT,
    atributo1   TEXT      NOT NULL,
    id_padre    INTEGER NOT NULL,
    FOREIGN KEY (id_padre) REFERENCES entidad_padre(id_padre)
)''')

# TABLA INTERMEDIA: relación N:M
cur.execute('''
CREATE TABLE IF NOT EXISTS tabla_union (
    id_a        INTEGER NOT NULL,
    id_b        INTEGER NOT NULL,
    atrib_rel   TEXT,
    PRIMARY KEY (id_a, id_b),
    FOREIGN KEY (id_a) REFERENCES entidad_a(id_a),
    FOREIGN KEY (id_b) REFERENCES entidad_b(id_b)
)''')
conn.commit()
```

Paso 4 — Insertar datos de prueba

```
# Mínimo requerido por tabla: 5 filas
# Los datos deben ser coherentes con el contexto del proyecto
# Ejemplo para proyecto de detección de fraude Interbank:
INSERT INTO clientes (nombre, dni, email, ciudad) VALUES
('Maria Quispe Ttito', '40123456', 'mquispe@bcp.pe', 'Cusco'),
('Carlos Huanca', '41234567', 'chuanca@gmail.com', 'Cusco'),
... (minimo 5 filas)
conn.commit()
print('Datos de prueba insertados')
```

08 Checklist de entrega del Lab D4

#	Entregable	Puntos
1	Diagrama ER completo (Lucidchart, draw.io o papel fotografiado) Entidades, atributos, relaciones y cardinalidades claramente indicadas	25
2	Documento de normalización Justificación de 1FN, 2FN y 3FN aplicadas al modelo	20
3	Notebook de Google Colab (.ipynb) Celda de conexión, CREATE TABLE comentado, INSERT de datos de prueba	30
4	Consulta SQL de verificación Al menos un SELECT con JOIN que cruce las tablas principales	15
5	Presentación en clase (5 minutos por grupo) Explicar el modelo ER y demostrar el Colab funcionando	10

Errores frecuentes que cuestan puntos:

- × Crear el Notebook pero no ejecutarlo — las celdas deben mostrar resultados.
- × Datos de prueba genericos ("nombre1", "test") — deben ser datos del contexto real.
- × FK sin la tabla padre creada primero — el orden de CREATE TABLE es obligatorio.
- × No indicar cardinalidades en el diagrama ER (1:N, N:M).
- × Mezclar atributos de distintas entidades en una sola tabla (viola 2FN o 3FN).

Consejos para un Lab D4 excelente:

- ☐ Usa nombres descriptivos: "id_cliente", "fecha_pedido" (no "id", "f1").
- ☐ Comenta cada línea de CREATE TABLE explicando qué hace.
- ☐ Verifica con SELECT * FROM cada tabla que los datos se insertaron bien.
- ☐ Usa PRAGMA foreign_keys = ON; en SQLite para activar la integridad referencial.
- ☐ El diagrama ER debe coincidir exactamente con las tablas que creas en Colab.